SmartWaste AI — Task 2

README and Python Package Requirements

An image-classification API that predicts one of six waste categories from an uploaded image. The model uses transfer learning with MobileNetV2 and is served through FastAPI.

1. Project objective

The API accepts a waste image and returns the predicted category, confidence score, and scores for six classes: **cardboard, glass, metal, paper, plastic, trash**.

2. Model and dataset

Model: ImageNet-pretrained MobileNetV2 with a custom classification head. **Input:** RGB image resized to 224 × 224. **Output:** six class scores. **Preprocessing:** `mobilenet_v2.preprocess_input`. The notebook uses a stratified 70% / 15% / 15% train/validation/test split. Use only one copy of the dataset to avoid duplicate-image leakage.

3. Packages and purpose

Package	Purpose
tensorflow	Model training/loading, preprocessing, and inference
numpy	Image arrays and prediction scores
Pillow	Open, convert, and resize uploaded images
fastapi	Define API endpoints
uvicorn[standard]	Run the FastAPI server
python-multipart	Handle uploaded files
scikit-learn	Data splitting and evaluation metrics
matplotlib	Training curves and plots
seaborn	Optional confusion-matrix visualization
kaggle	Optional dataset download
requests	Test API endpoints

4. Suggested project structure

```
SmartWasteAI/
■■■ app.py
■■■ smartwaste_model.keras
■■■ requirements.txt
■■■ README.md
```

The model file may be large. In Colab, the API expects `/content/smartwaste_model.keras`; update `MODEL_PATH` if your model is elsewhere.

5. Install dependencies

Choose a Python version supported by your TensorFlow release and platform. Python 3.10 or 3.11 is a common starting point for local setup; Google Colab often already has TensorFlow installed.

```
python -m venv .venv
# Windows:
.venv\Scripts\activate
# macOS / Linux:
source .venv/bin/activate

python -m pip install --upgrade pip
pip install -r requirements.txt
```

For API-only inference, the core packages are tensorflow, numpy, Pillow, fastapi, uvicorn[standard], and python-multipart. The remaining packages support training, evaluation, dataset download, or testing.

6. Run the API

```
uvicorn app:app --host 0.0.0.0 --port 8000
```

Open `http://127.0.0.1:8000/` for the home endpoint, `/health` for health, and `/docs` for Swagger UI.

For Google Colab, run:

```
from google.colab.output import eval_js
proxy_url = eval_js("google.colab.kernel.proxyPort(8000)")
print(proxy_url + "/docs")
```

7. Test an image in Swagger UI

1. Open the /docs URL. 2. Expand **POST /predict**. 3. Click **Try it out**. 4. Choose a JPG, JPEG, or PNG image. 5. Click **Execute**. 6. Review the response and status code.

Example response below is illustrative only; actual values depend on the image and model:

```
{
  "predicted_category": "plastic",
  "confidence": 96.42,
  "probabilities": {
    "cardboard": 0.01, "glass": 0.01, "metal": 0.01,
    "paper": 0.01, "plastic": 0.9642, "trash": 0.0088
  }
}
```

8. API endpoints

Method	Endpoint	Purpose
GET	/	Basic API message and supported categories
GET	/health	Check whether service and model are loaded
POST	/predict	Upload an image and receive classification

9. Common issues

Connection refused: ensure Uvicorn is running and wait for startup. **HTTP 422:** verify an image was selected and the upload field matches app.py. **HTTP 500:** inspect server logs. **Model not found:** correct MODEL_PATH. **Unsupported image:** upload a valid JPG, JPEG, or PNG.

10. Security and deployment

Never publish Kaggle API tokens or secrets in notebooks or source control. Use Colab Secrets or environment variables. Public deployment should include request-size limits, robust image validation, and suitable access controls. A confidence score is not necessarily a calibrated probability.

11. requirements.txt

```
# SmartWaste AI — Task 2 dependencies
# Install with: python -m pip install -r requirements.txt
# TensorFlow versions vary by Python version and operating system.
# Google Colab usually already includes TensorFlow; avoid reinstalling unless needed.

tensorflow
numpy
Pillow
fastapi
uvicorn[standard]
python-multipart

# Training and evaluation
scikit-learn
matplotlib
seaborn

# Optional: Kaggle dataset download and API testing
kaggle
requests
```